

Blatt 03: Methodenrefs, Lambdas, Observer

Calculator: Anonyme Klassen und Lambda-Ausdrücke

Was ist @Override nochmal?

- @Override wird verwendet, um eine geerbte Methode aus einer Basisklasse (Superklasse) in einer Unterklasse neu zu definieren

Was ist eine Anonyme Klasse und wozu verwende ich sie?

- Eine anonyme Klasse ist eine namenlose Klasse, die in einem einzigen Schritt direkt deklariert und instanziiert (als Objekt erzeugt) wird. Sie existiert nicht als eigenständige Datei, sondern ist direkt in den Programmcode eingebettet. Da sie keinen Namen hat, kann sie auch nirgendwo anders im Code wiederverwendet oder erneut aufgerufen werden.
- Anonyme Klassen werden immer dann eingesetzt, wenn eine Klasse nur an einer einzigen Stelle im Programm benötigt wird.
- braucht immer einen Typ (Interface oder Klasse)

Was ist ein Lambda-Ausdruck?

- eine kurze, anonyme Funktion (eine Methode ohne Namen)
- **(Parameter) -> {Anweisungen;}**
- Lambda-Ausdrücke funktionieren nur in Verbindung mit *funktionalen Interfaces*. Das sind Schnittstellen, die exakt eine abstrakte Methode vorschreiben.
- Kann man statt einer anonymen Klasse verwenden

LockSnake: Lambda-Ausdrücke, Methodenreferenzen und Observer-Pattern

Was sind Methodenreferenzen und wie verwende ich sie?

Methodenreferenzen (::) in Java sind eine kompakte, lesbare Kurzschreibweise für Lambda-Ausdrücke. Sie werden verwendet, um eine bereits existierende Methode direkt als Implementierung eines funktionalen Interfaces aufzurufen. Sie machen den Code übersichtlicher, indem sie Wiederholungen reduzieren.

Was ist das Observer-Pattern?

Es definiert eine 1-zu-n-Abhängigkeit zwischen Objekten. Wenn sich der Zustand eines Objekts ändert, werden alle abhängigen Objekte automatisch darüber benachrichtigt und aktualisiert.

3 Lambdas:

1. `allSet = pins.stream().allMatch(p -> p.state().isSet());`
2. `observers.forEach(obs -> obs.onGameStateChanged(state));`
3. `Runnable r =
 () -> {
 notifyObserver();
 notifyObservers();
 };`

2 Methodenreferenzen:

1. `engine.addObserver(panel::onGameStateChanged);`
2. `List<Runnable> actions = List.of(this::notifyObserver, this::notifyObservers);
 actions.forEach(Runnable::run);`

10 Tests:

```
// 1. Early exit: Status nicht RUNNING
// 2. Early exit: pendingDirection == NONE
// 3. LOST_OUT_OF_BOUNDS
// 4. Wand blockiert Bewegung
// 5. Beißt sich
// 6. Pin blockiert, wenn HIGH
// 7. Pin blockiert, wenn falsche Richtung
// 8. Pin wird gesetzt, wenn richtige Richtung
// 9. Schlange bewegt sich normal
// 10. Schlange beißt sich, wenn sie rückwärtsgeht
```